

『GUNDAM EVOLUTION』 エイムアシスト機能開発

1. プロジェクト基本情報

項目	内容
プロジェクト	『GUNDAM EVOLUTION』
プロジェクト概要	ゲームパッド操作向けエイムアシスト機能の開発
開発環境	Unreal Engine
開発言語	C++
主な技術	Unreal Engine Actor Component、Line Trace、Blueprint連携、プロファイリング統計
担当領域	企画仕样に基づくエイムアシスト機能の設計協議・実装

2. 開発背景・目的・担当範囲

2.1 開発背景と目的

シューティングゲームにおけるゲームパッドでの照準操作では、スティック入力の特性上、マウス操作と比較して細かな照準合わせが難しくなります。一方、補正が強すぎる場合はプレイヤーの意図を妨げ、操作感を損なう可能性があります。

本機能では、ゲームパッドでの照準操作を補助し、プレイヤーの意図を損なわずに狙いやすさを向上させることを目的としました。補助方式と操作感の方向性は企画担当者が策定し、私は協議を通じた仕様の具体化と実装を担当しました。

2.2 個人の担当範囲

私は、エイムアシスト機能を担うコンポーネントについて、企画担当者との仕様協議および技術設計・実装を担当しました。主な担当内容は以下の通りです。

- エイムアシスト機能全体の技術設計および実装
- 操作感の調整・検証を支援する機能の実装

3. 機能概要

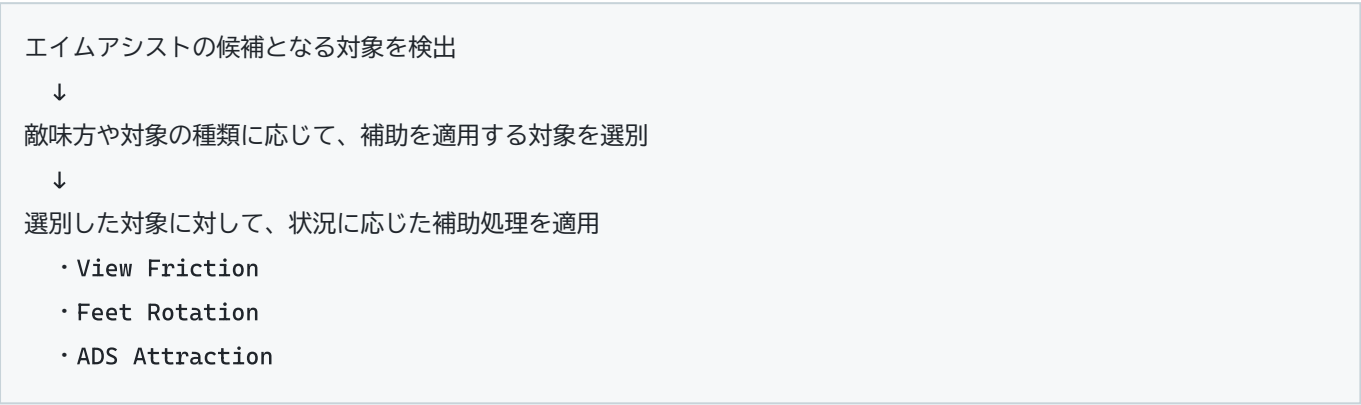
本機能は、役割の異なる次の3種類の補助処理で構成しました。

補助処理	役割
View Friction	対象付近で視点移動を緩やかにする
Feet Rotation	左右移動に伴う照準のずれを補正する

補助処理	役割
ADS Attraction	ADS開始時に照準を対象方向へ補正する

各処理は個別に有効化および調整できる構成としました。

4. 全体の処理構成



対象を検出・選別した後、プレイヤーの状態に応じて各補助処理を適用する構成としました。

5. 代表的な技術課題と解決方法

5.1 操作感を数理モデルへ落とし込む設計

課題

企画担当者が意図する「狙いやすさ」や「自然な補助」は感覚的な要件であり、そのままではプログラムとして実装できません。プレイヤーの操作を妨げずに再現するため、操作感を計算可能な要素へ置き換える必要がありました。

判断

対象までの距離、照準位置、移動方向・速度、対象との角度差、補助時間などへ要件を分解し、補助処理ごとに補正量を求める数理モデルを設計しました。

実施内容

- ・View Frictionでは、距離と照準位置に応じた係数によって視点入力を減衰させました
- ・Feet Rotationでは、移動方向・速度と対象との位置関係から照準の補正量を算出しました
- ・ADS Attractionでは、カメラと対象の角度差から補正量を算出し、適用時間やプレイヤー入力に応じて制御しました
- ・各モデルの強度と適用範囲をパラメーター化し、確認結果に応じて調整できるようにしました

結果

感覚的な操作要件を再現可能な計算処理として実装し、企画担当者やQA担当者の確認結果を数値調整へ反映できるようにしました。

5.2 調整・デバッグ・性能確認を考慮した実装

課題

操作感を調整するための動作確認と、処理負荷を把握するための計測手段が必要でした。

実施内容

- 補助処理ごとの実行時切り替えを用意しました
- 判定値や対象との位置関係を確認できるデバッグ表示を実装しました
- 処理単位で負荷を確認できる計測機能を組み込みました

結果

操作感の調整と問題の切り分けを行いやすい環境を整えました。

6. 検証方法

実装後の操作感と機能の妥当性については、開発責任者およびQA担当者が実機上で確認しました。確認結果と指摘内容を基に、企画意図とのずれや操作感に関する課題を共有し、必要な調整へ反映しました。

また、実装側では、補助方式ごとの実行時切り替え、対象との距離・角度・補正係数の画面表示、デバッグライン、処理区間別のプロファイリング統計を用意しました。これにより、責任者およびQA担当者から指摘された現象について、補助方式ごとに条件を切り分け、対象判定と補正計算の状態を確認できるようにしました。

7. 成果

- ゲームパッド操作向けに、3種類のエイムアシスト処理を実装しました
- 操作感を数値で調整できる構成とし、開発責任者およびQA担当者の確認を経て製品へ導入しました

8. 本プロジェクトで得た知識・能力

8.1 操作感を数値へ分解して実装する能力

「照準を合わせやすくする」という感覚的な要件を、距離、角度、入力値、移動速度、時間、適用範囲といった計算可能な要素へ分解しました。その上で、目的の異なる補助処理を組み合わせ、プレイヤー入力を基準とした補正として実装する経験を得ました。

8.2 Unreal EngineにおけるGameplay機能の実装

Actor Component、Camera Component、PlayerController、Line Traceなどを用いた実装を通して、Unreal EngineのGameplay機能に関する理解を深めました。

8.3 エイムアシスト機能の設計・実装に関する知識

各補助処理の役割に加え、対象の検出・選別、補正量の計算、パラメーター調整まで、エイムアシスト機能を設計・実装するための知識を得ました。

9. 企画担当者との共同開発と関連公開資料

本機能は、企画担当者が策定した補助方式と操作感の方向性を基に、仕様協議を重ねながら開発しました。

企画担当者は、実現したい操作感、各補助方式の役割、適用条件および調整方針を整理しました。私はそれらの要件について技術的な実現方法を検討し、対象検出・選別、補正量の計算、パラメーター構成、デバッグ・性能確認機能を設計・実装しました。

共同開発した企画担当者が、エイムアシストの代表的な方式と設計上の考え方について、以下の記事を公開しています。

- [Aim Assist Types](https://medium.com/@hyang0451/aim-assist-types-6c5993d47a10) (https://medium.com/@hyang0451/aim-assist-types-6c5993d47a10)
- 執筆者：本機能を共同開発した企画担当者

この記事は企画担当者本人が執筆したものであり、私の著作物ではありません。本資料では、企画要件をどのように計算可能なモデルへ分解し、ゲーム内の機能として設計・実装したかという、私自身の担当範囲を説明しています。また、リンク先の記事は関連する公開技術資料であり、記事中の説明と製品版の具体的な実装がすべて同一であることを示すものではありません。